

Présentation du projet

Application web de recherche de trajets en TGV

Garcia Jean Claude

Soutenance à Versailles le 26/06/2026

Contexte

- Le client souhaite une application web permettant de rechercher un trajet en train entre deux villes françaises.
- Le site doit être simple, accessible et adapté à tous les profils d'utilisateurs.
- L'application doit fonctionner sur mobile, tablette et ordinateur. Une API et une base de données sont nécessaires pour gérer les données dynamiquement.

Objectifs fonctionnels

- Permettre à l'utilisateur de se connecter.
- Choisir une ville de départ.
- Choisir une ville d'arrivée.
- Lancer une recherche de trajet.
- Consulter les villes, les lignes et les horaires.
- Proposer une interface en français et en anglais
- Proposer plusieurs palettes de couleurs.

Fonctionnement de l'application

1. L'utilisateur se connecte.
2. Il choisit une ville de départ.
3. Il choisit une ville d'arrivée.
4. Il lance la recherche
5. L'application affiche les trajets disponibles

Technologies utilisées

- Front-end : ReactJS, Bootstrap, Sass.
- Back-end : Node.js, Express.
- Base de données : MySQL.
- ORM : Sequelize.
- Versioning : Git et GitHub.
- Maquettage : Figma.

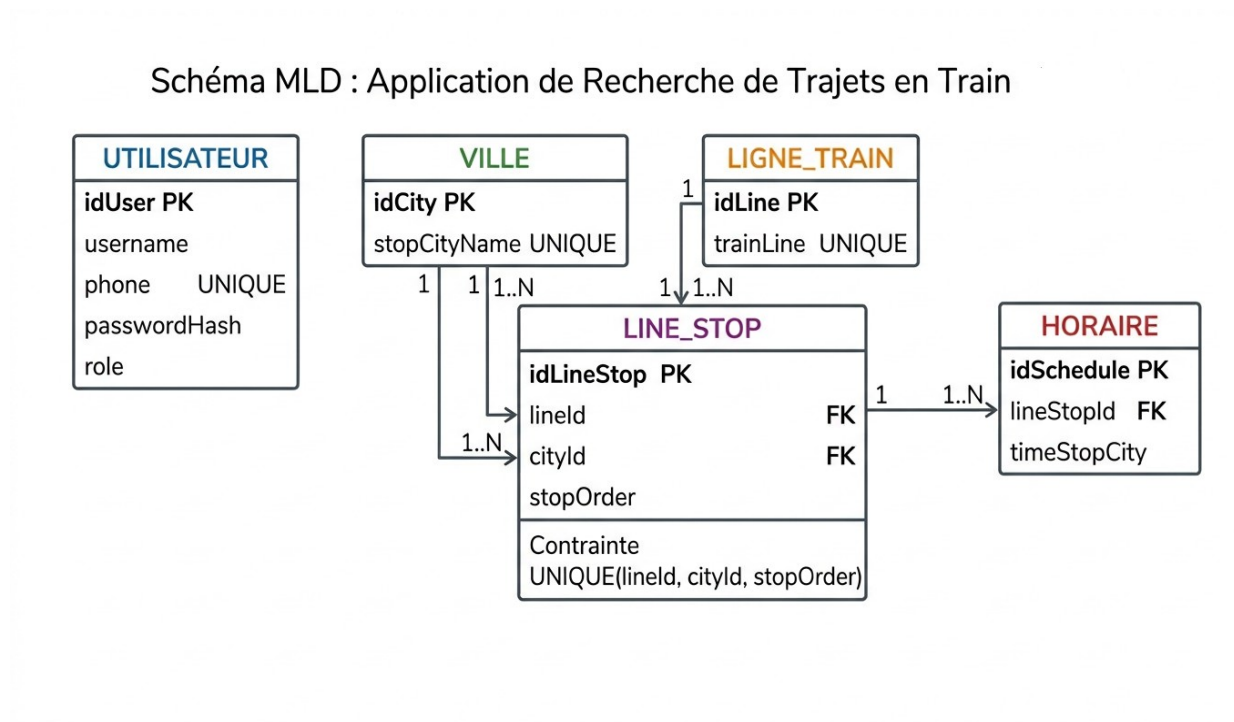
Architecture du projet

- Une application front-end en React.
- Une API REST développée avec Express.
- Une base de données MySQL uniquement interrogée par l'API.
- Une authentification sécurisée par jeton JWT.
- Une séparation claire entre interface, logique métier et données.

Modélisation des données

- Utilisateur : username, téléphone, mot de passe, rôle
- Ville : nom de la ville.
- Ligne de train : nom de la ligne.
- Arrêt : association entre une ligne et une ville.
- Horaire : heure de passage à un arrêt.

BDD – MLD (Modèle Logique de Données – textuel)



MCD – Application de recherche de trajets de trains

➤ UTILISATEUR

- idUser (PK)
- username (unique)
- phone
- passwordHash
- role (user | admin)

➤ TRAINLINE

- idLine (PK)
- trainLine

Mon train

➤ CITY

- idCity (PK)
- nameCity

➤ LINESTOP (association)

Représente un arrêt dans une ligne avec son ordre

- idLineStop (PK)
- trainLineId (FK)
- cityId (FK)
- stopOrder

Cardinalités :

Une TRAINLINE possède 1,N LINESTOP

Une CITY apparaît dans 1,N LINESTOP

➤ SCHEDULE

Horaire d'un bus à un arrêt précis

- idTime (PK)
- lineStopId (FK)
- time

Cardinalités :

Un LINESTOP possède 0,N SCHEDULE

Un SCHEDULE appartient à 1,1 LINESTOP

Relations (cardinalités)

UTILISATEUR

→ aucune relation métier avec les autres tables (auth seulement)

TRAINLINE (1,N) — contient — LINESTOP (1,1)

CITY (1,N) — est desservie par — LINESTOP (1,1)

LINE_STOP (1,1) — possède — SCHEDULE (0,N)

Relations :

TRAIN_LINE (1,N) — LINE_STOP — (N,1) CITY

LINE_STOP (1,1) — (0,N) SCHEDULE

Sécurité et accès

- Mots de passe hashés en base.
- Authentification par JWT.
- Contrôle d'accès selon le rôle utilisateur.
- API limitée à l'application front-end.
- Validation des données côté serveur.
- Configuration CORS adaptée.

Accessibilité et responsive

- Respect des principes WCAG 2.1.
- Contrastes de couleurs adaptés.
- Labels explicites sur les formulaires.
- Design mobile first.
- Interface responsive sur tous les écrans.

Bilan

- Projet fonctionnel et structuré.
- Front-end, API et base de données séparés.
- Contraintes d'accessibilité et de sécurité intégrées.
- Projet évolutif pour de futures fonctionnalités.